

Get Ready for Your
Interview with These Key
Git Interview Questions!



Top Git Interview Questions and Answers for Freshers

Git Interview Questions

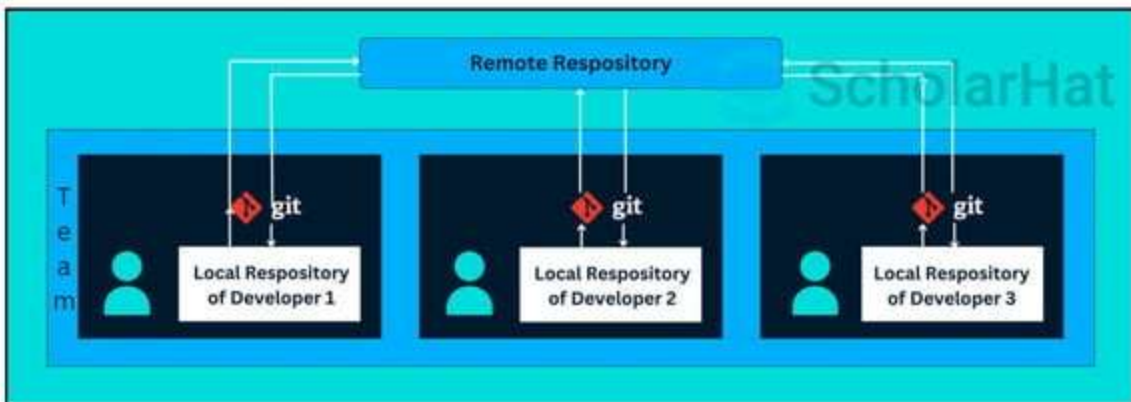
Git is one of the most widely used version control systems in the **software development** industry. If you're preparing for a **Git interview**, understanding the most commonly asked questions can help you stand out. **Git** is essential for collaboration and tracking changes in code, so it's important to master its features and commands.

In this [Interview tutorial](#), we'll cover a list of typical **Git interview questions** and provide insights into the concepts you need to focus on. Whether you're just starting with Git or have prior experience, this guide will help you prepare for your next **Git interview**.

What is Git, and why is it used?

Git is the most popular **open-source version control system**. It helps you manage both small and large projects in an efficient and organized way. **Git** is perfect for teams, allowing easy

tracking of changes and smooth collaboration. Isn't it great to know exactly who made which changes and when? With Git, you can **identify bugs**, **run tests**, and implement **new features**. If something goes wrong, you can easily revert to a previous working version, saving time and effort. Isn't it reassuring to have that kind of control over your code?



Preparation Tips to Crack the Git Interview

Preparing for a Git interview? Here are some tips to help you stand out:

1. Understand Core Concepts

- **Learn the Basics:** Make sure you understand fundamental concepts like **version control** and the difference between **Git** and other **version control systems (VCS)**. This knowledge is essential in **software development**.
- **Master Git Commands:** Get comfortable with common **Git** commands like **git init**, **git clone**, **git add**, **git commit**, **git push**, **git pull**, **git branch**, **git checkout**, **git merge**, and **git rebase**. Understanding what each command does and when to use it is crucial.
- **Know Git Workflows:** Familiarize yourself with workflows like **Git Flow**, **GitHub Flow**, and **GitLab Flow**. Understand the pros and cons of each and when to use them.

2. Hands-on Practice

- **Use Git Regularly:** Practice using **Git** on your personal or professional projects. The more you use it, the more comfortable you'll become with different **commands** and **workflows**.
- **Experiment with Branching and Merging:** Create **branches**, **merge** them, and **resolve conflicts**. Knowing how to manage branches and merge them smoothly is essential.
- **Contribute to Open Source Projects:** Engage with open source projects on platforms like **GitHub**. This will give you practical experience with collaboration tools and workflows used

in the industry.

3. Dive into Advanced Topics

- **Understand Git Internals:** Learn about the **.git directory**, the **staging area**, and how **Git stores** changes. This knowledge will give you an edge over other candidates.
- **Learn Advanced Commands:** Get familiar with advanced commands like **git stash**, **git rebase --interactive**, **git bisect**, **git cherry-pick**, and using hooks.
- **Explore Git Tools and Extensions:** Familiarize yourself with tools like **Git LFS** (Large File Storage), **Git GUI clients**, and **IDE integrations** to enhance your **Git workflow**.

4. Review and Understand Common Errors

- **Familiarize with Common Issues:** Understand common problems like **merge conflicts**, **detached HEAD state**, and how to revert changes or **recover lost commits**.
- **Practice Troubleshooting:** It's not enough to avoid issues; knowing how to diagnose and fix them is just as important. Practice troubleshooting common **Git scenarios**.

5. Prepare for Behavioral Questions

- **Collaboration Scenarios:** Be ready to share how you've used **Git** in team settings, handled merge conflicts, and improved collaboration using Git in your projects.
- **Contribution Examples:** Highlight specific contributions you've made to projects using Git, showing your understanding of best practices in version control.

6. Stay Updated

- **Follow Git Blogs:** Stay informed about the latest **Git features** and best practices by following relevant blogs and development communities.
- **Explore Git Documentation:** The official **Git documentation** is an excellent resource to deepen your knowledge and keep up with new features.

7. Mock Interviews

- **Practice with Peers or Mentors:** Conduct mock interviews to get comfortable articulating your Git knowledge and experience.
- **Use Online Platforms:** Platforms like Pramp or LeetCode offer mock interviews and problem-solving exercises that can help you prepare for the real interview.

8. Review Interview Questions

- **Study Common Questions:** Review common Git interview questions, both theoretical and practical, and prepare your answers.

9. Showcase Your Knowledge

- **Create a Portfolio:** Have a portfolio of projects or contributions to showcase your practical experience with Git. A GitHub profile or personal website can help you demonstrate your skills.

Git Interview Questions and Answers for Freshers

1. What is Git?

Git is an open-source distributed version control system. It helps you manage your code efficiently, track changes, and collaborate with your team seamlessly. It ensures that you can work on projects without overwriting anyone's work. Isn't it amazing to have this kind of control?

2. How is Git different from other version control systems?

Git is distributed, meaning every user has a complete copy of the repository. Unlike centralized systems, it doesn't rely on a single server, making it faster and more resilient. Don't you love its reliability?

3. Explain the difference between Git and GitHub.

Git is a version control tool, while **GitHub** is a platform for hosting Git repositories. Think of Git as your tool to manage code and GitHub as a place where you share and collaborate on that code. Sounds simple, right?

4. What are some essential Git commands?

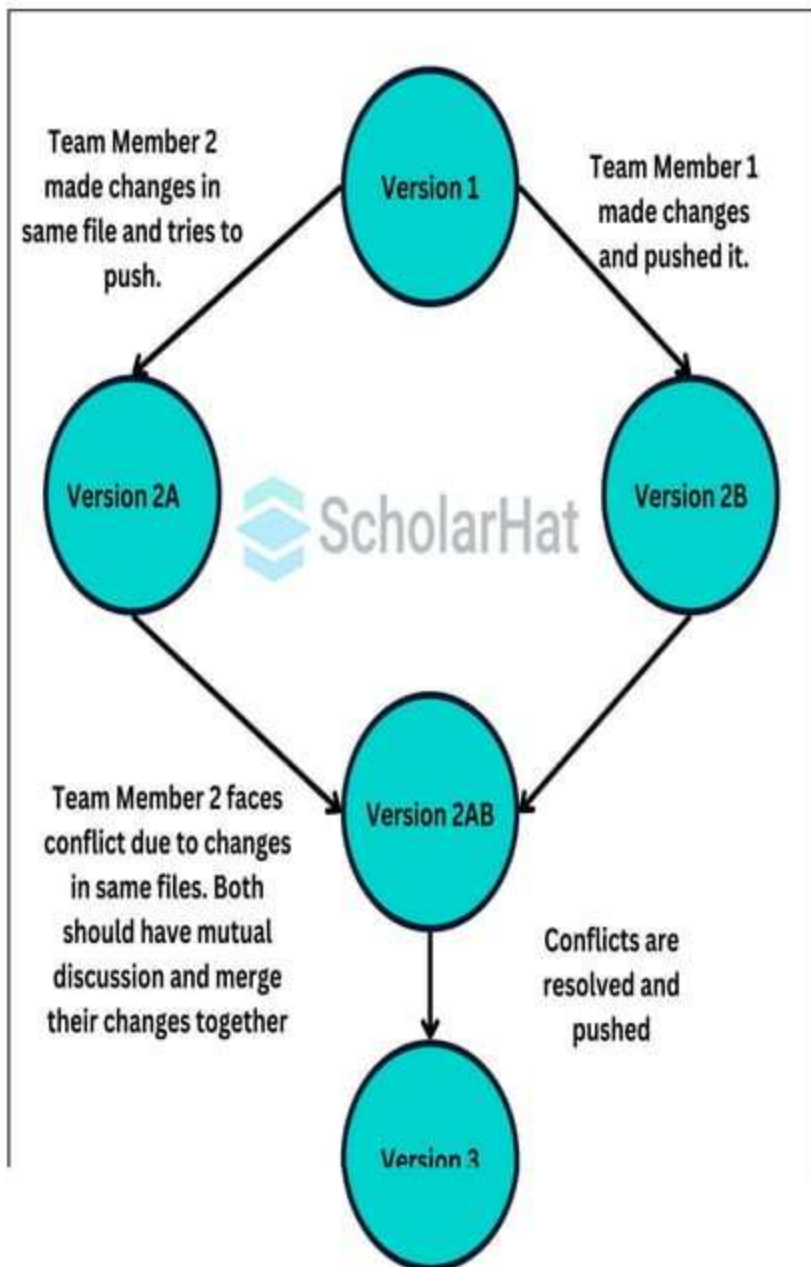
Here's a list of common commands you should know:

- **git init:** Initializes a Git repository.
- **git clone:** Clones a repository.
- **git add:** Stages changes.
- **git commit:** Saves staged changes.

- `git push`: Uploads changes to the remote repository.
- `git pull`: Fetches and merges changes from the remote repository.

5. What is a conflict in Git?

A **conflict** in Git happens when it cannot automatically merge changes from different branches. This often occurs in team environments. Common conflict scenarios include:



- When two branches modify the **same line** in a file.
- When one branch **deletes a file**, but the other branch modifies it.

Git cannot decide which changes to prioritize in these cases. You need to **resolve conflicts manually** by reviewing the changes and collaborating with your team to determine the best outcome.

6. How do you create a new Git repository?

To create a repository, run the following command in your project folder:

```
git init
```

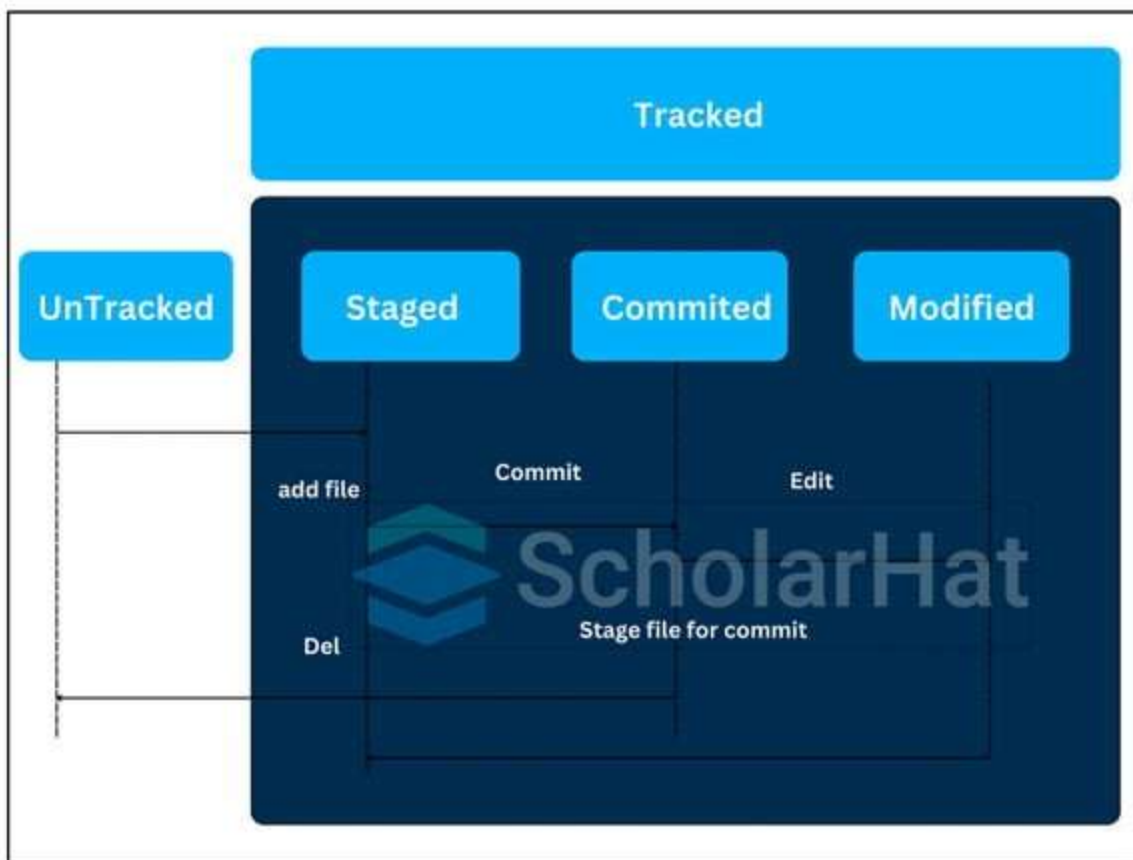


This initializes a Git repository in your project directory.

7. What is the purpose of the .git directory?

The **.git** directory stores all the metadata and history of your repository. Without it, your project wouldn't be tracked by Git. It's like the brain of your repository.

8. Define "Index" in Git



The **Index**, also known as the **Staging Area**, is where changes are placed before they are committed in **Git**. It allows the developer to review, format, and make adjustments to the files before finalizing the commit. This area serves as a buffer between the working directory and the repository, giving you the ability to organize and fine-tune changes before they are saved permanently.

9. What is a branch in Git?

A **branch** in **Git** is like a parallel line of development. It lets you work on new features without affecting the main codebase. You can merge the branch once your work is complete. Isn't that a neat way to stay organized?

10. How do you create a branch in Git?

Use the following command to create a branch:

```
git branch branch-name
```



To switch to the branch, use:

```
git checkout branch-name
```



11. How do you merge branches in Git?

Switch to the branch you want to merge into and run:

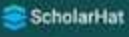
```
git merge branch-name
```



This merges the specified branch into your current branch.



Become A Full-Stack .NET Developer - SDE

 ScholarHat

-  Build End-to-end Project
-  90% Hands-on Learning
-  Get a Software Job of upto ₹9 LPA*
-  Hands-On Labs to Build Confidence
-  Interview Prep

Book A Live Demo & Get 30% OFF



12. What is a commit in Git?

A **commit** is like a **snapshot** of your project. It records changes you've made and is a critical part of **version control**. Don't you think it's cool to keep track of every change?

13. How do you view the commit history?

Run the following command to see the commit history:

```
git log
```



14. What is a merge conflict?

A **merge conflict** happens when **Git** cannot automatically combine changes from two branches. You need to resolve the conflict manually before proceeding. Isn't it good to have

control over such situations?

15. How do you resolve merge conflicts?

When a **conflict** occurs, **Git** marks the conflicting sections in the affected files. You need to manually edit the files, keeping or modifying the desired changes, and then stage the resolved files using **git add**.

16. What is a detached HEAD state in Git?

A **detached HEAD** state occurs when you're not on a branch but on a specific commit. To get back to a branch, use:

```
git checkout branch-name
```



17. How do you stash changes in Git?

If you want to save changes without committing them, use:

```
git stash
```



To apply stashed changes later, use:

```
git stash apply
```



18. What is Git rebase?

Rebasing moves your branch to the tip of another branch, rewriting the commit history. It's great for creating a linear history but should be used carefully. Don't you think it's powerful?

19. How do you remove a file from Git tracking?

To remove a file but keep it in your working directory, use:

```
git rm --cached file-name
```



20. How do you revert a commit in Git?

To revert a specific commit, use:

```
git revert commit-hash
```



This creates a new commit that undoes the changes from the specified commit.

Git Interview Questions and Answers for Intermediates

21. What is the difference between `git reset` and `git revert`?

git reset undoes changes by moving the branch pointer to a previous commit, and it can alter history. **git revert**, on the other hand, creates a new commit that reverses changes from a specific commit. Isn't it safer to use **git revert** in shared repositories?

22. How do you squash commits in Git?

You can **squash commits** by using an interactive rebase:

```
git rebase -i HEAD~n
```



Replace `n` with the number of commits you want to squash. Combine commits into one by marking subsequent commits as `squash` or `s`.

23. What is the purpose of the `.gitignore` file?

The `.gitignore` file tells Git which files or directories to ignore. For example, you might not want to track log files or build artifacts. Doesn't it simplify your workflow?

24. How do you configure a Git alias?

You can set a **Git alias** to save time. For example, to shorten `git status`, you can use:

```
git config --global alias.st status
```



25. What are Git tags, and how do you create one?

Tags are used to mark specific points in history, like releases. To create a tag, use:

```
git tag tag-name
```

To push the tag to a remote repository:

```
git push origin tag-name
```

26. What is the difference between annotated and lightweight tags?

Annotated tags store metadata like the tagger's name, email, and date. **Lightweight tags** are simple pointers to a commit. Annotated tags are preferred for releases since they carry more information. Isn't that a useful distinction?

27. How do you undo a pushed commit?

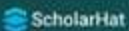
If you want to undo a commit that has been pushed, you can revert it with:

```
git revert commit-hash
```

This ensures a safe history, especially in shared repositories.



Become A Full-Stack .NET Developer - SDE



 Build End-to-end Project

 90% Hands-on Learning

 Get a Software Job of upto ₹9 LPA*

 Hands-On Labs to Build Confidence

 Interview Prep



[Book A Live Demo & Get 30% OFF](#)

28. What are Git hooks, and how do you use them?

Git hooks are custom scripts that run during certain events like commits or merges. For example, you can create a pre-commit hook to check for linting errors. Isn't it handy for automating tasks?

29. How do you cherry-pick a commit in Git?

If you want to apply a specific commit from another branch, use:

```
git cherry-pick commit-hash
```



This copies the commit to your current branch.

30. What is Git reflog?

Reflog tracks updates to your repository's references, such as branch changes or resets. To view the reflog, use:

```
git reflog
```



It helps you recover commits that seem lost. Isn't that reassuring?

31. How do you perform an interactive rebase?

Use the following command for an interactive rebase:

```
git rebase -i base-branch
```



You can reorder, squash, or edit commits as needed.

32. What is the difference between `git fetch` and `git pull`?

git fetch downloads changes from the remote repository but doesn't merge them. **git pull** fetches and merges changes. Don't you think it's better to use **git fetch** for more control?

33. How do you delete a branch in Git?

To delete a local branch, use:

```
git branch -d branch-name
```



To delete a remote branch, use:

```
git push origin --delete branch-name
```



34. What is a submodule in Git?

A **submodule** is a repository within a repository. It allows you to include another repository as a dependency. Use the following command to add a submodule:

```
git submodule add repository-url
```



35. How do you handle large files in Git?

To manage large files, you can use **Git LFS (Large File Storage)**. Install it and track specific files:

```
git lfs track "*.filetype"
```



This prevents large files from slowing down your repository.

Git Interview Questions and Answers for Experienced

36. How do you handle conflicts during a **git rebase**?

When conflicts arise during a **rebase**, **Git** pauses and lets you resolve them. After resolving, use:

```
git add resolved-file
```



Then continue the rebase with:

```
git rebase --continue
```



Isn't it crucial to test thoroughly after resolving conflicts?

37. How do you optimize a Git repository for performance?

You can optimize your repository by removing unnecessary data with:

```
git gc --aggressive --prune=now
```



This cleans up loose objects and compresses repository data for better performance.

38. What are Git hooks, and how can they be used in CI/CD pipelines?

Git hooks are scripts triggered by specific events. For CI/CD, use hooks like **post-commit** or **pre-push** to trigger build and deployment pipelines. Have you automated your workflows this way?

39. How do you recover a deleted branch in Git?

If the branch was deleted but its commits are still referenced, you can recover it using the **reflog**:

```
git reflog
```

Find the commit hash and create the branch again:



The advertisement features a dark blue background with a light blue 'NET Core' logo on the left. The main headline reads 'Become A Full-Stack .NET Developer - SDE'. Below this, there are four bullet points: 'Build End-to-end Project' with a rocket icon, '90% Hands-on Learning' with a code icon, 'Hands-On Labs to Build Confidence' with a flask icon, and 'Interview Prep' with a question mark icon. A fifth bullet point at the bottom left says 'Get a Software Job of upto ₹9 LPA*' with a money bag icon. On the right side, there is a photo of a man in a suit. At the bottom right, a white button with black text says 'Book A Live Demo & Get 30% OFF'. The ScholarHat logo is in the top right corner.

```
git checkout -b branch-name commit-hash
```

40. What is the difference between shallow cloning and full cloning?

Shallow cloning clones only the latest commits, using:

```
git clone --depth=1 repo-url
```

It's faster but doesn't include full history, unlike full cloning. Isn't it useful for large repositories?

41. How do you manage binary files in Git?

For binary files, use **Git LFS** to track them efficiently. Install and configure it with:

```
git lfs install
```

Track specific file types:

```
git lfs track "*.filetype"
```

42. What is the use of `git bundle`?

git bundle creates a single file containing repository data, which can be shared without remote access. Create a bundle with:

```
git bundle create bundle-name branch-name
```

Then clone it with:

```
git clone bundle-name
```

43. How do you ensure Git history remains clean?

Use interactive rebases to **squash commits** and follow branch naming conventions. Wouldn't a clean history make debugging easier?

44. What is the difference between `git rebase` and `git merge`?

git rebase integrates changes by rewriting commit history. **git merge** combines changes by creating a merge commit. **Rebasing** produces a linear history while merging preserves branch divergence. Isn't rebasing ideal for feature branches?

45. How do you remove sensitive data from Git history?

Use **git filter-repo** or **BFG Repo-Cleaner** to remove sensitive data. For example:

```
bfg --delete-files filename
```

46. How do you handle submodules in Git?

To initialize and update **submodules**, use:

```
git submodule update --init
```

Have you considered using **submodules** for shared libraries?

47. How do you debug a broken repository?

Use commands like:

```
git fsck
```

to identify corrupted objects. Isn't it essential to resolve repository issues quickly?

48. How do you revert multiple commits in Git?

To revert multiple commits, use:

```
git revert first-commit..last-commit
```

49. How do you enforce Git commit message standards?

Use a **commit-msg hook** to validate commit messages. For example:

```
#!/bin/sh
if ! grep -q 'JIRA-[d\+]' "$1"; then
  echo "Commit message must include a JIRA ticket."
  exit 1
fi
```

50. How do you manage large teams in Git?

Use protected branches, enforce code reviews, and adopt a workflow like **Git Flow** or **GitHub Flow**. Don't you think these practices enhance collaboration?

Summary

Git is a **distributed version control system** that allows multiple developers to work on a project simultaneously. It helps in tracking changes, managing **source code** history, and **collaborating** on code. Git is widely used for **managing** and **sharing** code, enabling teams to work more efficiently. Interested in mastering Git and becoming an expert in **version control**? Check out the [Scholarhat Azure DevOps Training | Azure DevOps Certification Course](#) for hands-on practice and real-world projects.

Additionally, explore the [Docker and Kubernetes Certification Training](#) to gain a deeper understanding of containerization, orchestration, and how they integrate with Git for efficient DevOps workflows.

Exclusive Free Courses

Scholarhat offers amazing [free courses](#) to help you build job-ready skills and boost your career. Check them out below:

- [Free Azure Fundamentals Course](#) – Learn the fundamentals of Git and its version control integration.
- [Free Docker Certification Course](#) – Understand containerization and Docker integration with Git.

Enhance your skills today and become an expert in Git and version control!

Test Your Knowledge of Git!

Q 1: What is Git?

- (a) A text editor
- (b) A version control system
- (c) A project management tool
- (d) A build automation tool

View Answer ↓

Q 2: What is a Git repository?

- (a) A place to store code
- (b) A tool for automating code build

- (c) A place to store and track code changes
- (d) A code-testing tool

View Answer ↓

Q 3: What is a commit in Git?

- (a) A tool for merging code
- (b) A snapshot of changes made to the repository
- (c) A branch in the Git repository
- (d) A version control system

View Answer ↓

Q 4: What is the purpose of branching in Git?

- (a) To merge code
- (b) To separate different versions of code
- (c) To delete code
- (d) To track project changes

View Answer ↓

Q 5: Which Git command is used to clone a repository?

- (a) git fetch
- (b) git pull
- (c) git clone
- (d) git push

View Answer ↓